Sparse(r) Loki

Purva Chiniya
University of Maryland
pchiniya@umd.edu

Connor Dilgren
University of Maryland
cdilgren@umd.edu

Sukriti Paul
University of Maryland
sukriti5@umd.edu

Abstract

This paper presents three sparsity methods that can be used in conjunction with Loki, an approximate attention method. We evaluate each method on a microbenchmark modeled on the Llama-2-13B model, and show that using a fixed sparsity pattern as a pre-filtering step on the key matrix results in a 5-15% speedup, and enforcing a 2:4 fine-grained structured sparsity pattern on the weight matrices results in a 10-15% speedup. We also show that applying unstructured sparsity patterns that do not utilize built-in NVIDIA hardware accelerations do not result in any speedup. Finally, we show that 2:4 fine-grained structured sparsity can improve downstream performance over Loki by itself.

1 Introduction

Large Language Models (LLMs) have become integral for processing extensive sequences such as code repositories, lengthy documents, and legal reports. However, maintaining efficient inference performance grows increasingly challenging as the input sequence lengthens, primarily because the computational complexity of the self-attention mechanism scales quadratically with sequence length, significantly impacting inference speed.

To address this, various approximate attention methods have been proposed, sacrificing some accuracy to achieve better computational performance. One such promising technique is Loki [12], which employs a low-rank approximation approach, significantly reducing memory overhead. Based on principal component analysis (PCA) showing that key vectors occupy a lower dimension space than the full dimension of the attention head, this method calculates approximate attention scores in shared memory (SM) during inference using only the top `d` dimensions selected by PCA. The top-k most relevant keys are selected using these approximate attention scores, and then the final attention scores are computed in the full-dimension space using only the selected keys. Empirical evaluations show that Loki can accelerate attention computation by up to 45% in practice and achieve a theoretical speedup of up to 2.6x, albeit at the cost of storing the PCA transform matrix and minor performance degradation, with no associated KV-cache memory savings.

Given that Loki operates orthogonally to other sparsification methods, combining it with additional approaches could potentially amplify inference efficiency without significant performance trade-offs. In this paper, we explore the computational and downstream performance of Loki when combined with three additional sparsity-based optimization

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

techniques: Sparse Transformers [3], unstructured magnitude-based pruning in the query-key projection weights, and 2:4 fine-grained structured sparsity applied to all projection weights. Our evaluations, conducted on multiple benchmarks including MMLU [7], GPQA [11], GSM8k [5], and HotPotQA [14], indicate that integrating Loki with 2:4 structured sparsity not only preserves model accuracy but can also yield performance improvements of 1-3.2 percentage points on reasoning tasks, thus achieving superior balance between efficiency and accuracy.

2 Related Works

Sparse Attention Patterns Sparse attention mechanisms focus on reducing the quadratic complexity of self-attention by restricting token interactions through fixed or adaptive patterns. Seminal works such as Sparse Transformer [3], Longformer [1], and BigBird [17] propose algorithmic designs combining local, global, and random attention patterns to efficiently model long sequences. Recent studies [15] also explore predicting sparsity patterns dynamically to further optimize computation.

Unstructured and Structured Sparsity From a hardware and model compression perspective, sparsity is often categorized as unstructured or structured based on the regularity of zero patterns. Unstructured sparsity, such as magnitude pruning of attention projection weights [8], can reduce parameter counts but suffers from irregular memory access, limiting practical speedups. Structured sparsity, exemplified by block or windowed attention patterns in Sparse Transformer [3], Longformer [1], and BigBird [17], aligns well with hardware constraints and enables efficient acceleration. Comprehensive surveys [6] provide further insights into these sparsity modalities in transformer architectures.

Low-Rank and Hybrid Approaches Integrating low-rank approximations with sparsity has emerged as a potent strategy to enhance efficiency further. Methods such as Linformer [16] and Performer [4] employ low-rank approximations to reduce the memory and computational demands of the attention mechanism. Tay et al. [13] review various hybrid frameworks that incorporate sparse, low-rank, and kernel-based attention mechanisms, highlighting significant potential for compounded performance gains.

Summary Building on these insights, our work systematically evaluates the interplay between Loki, Sparse Transformers, and both structured and unstructured sparsity methods. By integrating low-rank and structured sparsity techniques, we address both the embedding and sequence-length dimensions of attention computation, aiming to advance efficient inference in large-scale transformer models.

3 Profiling Loki

To analyze the computational performance of Loki, we conducted detailed profiling using an attention benchmark representative of models such as Llama-2-13B, which features 32 attention heads and a hidden dimension of 128. Experiments were conducted on a single NVIDIA H100 GPU, with varying prompt lengths (512, 1024, 2048 tokens), generation steps (64, 128, 256 tokens), and utilizing top key percentages of 12.5% and 25%. For a fair and consistent comparison with our Loki modifications, we intentionally disabled Loki's optimized Triton kernels, as these kernels would require specific adaptation to align with our proposed modifications. Instead, native PyTorch operations were employed throughout.

Figure 1 summarizes the distribution of computational time across the various operations within Loki. We observe that 21% of the time on average is spent in the `qk-gen` step, which projects the input embedding to the key, query, and value vectors. This is not surprising, since Loki does not reduce the computational complexity of this operation. Thus, we evaluate methods that sparsify the Q, K, and V project matrices to reduce the computational burden.

Furthermore, we note significant computational overhead in three specific attention calculation steps: `qk-matmul-1` (16%), `top-k` (14%), and `qk-matmul-2` (19%). The `qk-matmul-1` step

computes approximate attention scores using the top PCA-transformed dimensions, the `top-k` step sorts these attention scores and selects the top `k` keys, and the `qk-matmul-2` step calculates full-dimensional attention scores using only the selected top-k keys. Since each of these operations can be sped up by reducing the number of keys used, we evaluate a fixed sparsity pattern that reduces the number of active keys before Loki is applied.

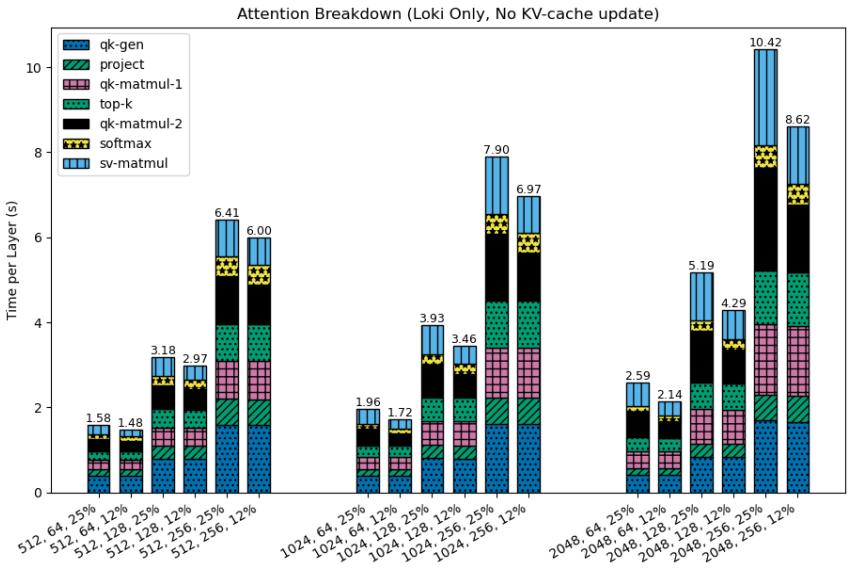

Figure 1: Average Time Per Layer for Loki operations (excluding the KV-cache update). The x-axis shows the benchmark settings for the prompt length, number of generation steps, and percent of keys to keep for the full attention calculation. All evaluations used the top 25% of the PCA-transformed key and query dimensions for the approximate attention score calculation.

4 Loki Sparsification Strategies

In this section, we detail three sparse attention methods that we integrated into Loki. Each of these methods is orthogonal to the Loki algorithm, and could be integrated into attention without Loki. To the best of our knowledge, this is the first time that these methods have been combined.

4.1 Sparse Transformers

One way to reduce the `qk-matmul-1`, `top-k`, `qk-matmul-2` times we observed in Figure 1 is to reduce the number of keys that Loki considers for its top-k calculation. We use Sparse Transformers [3] for this operation. Our implementation is based on the fixed-sparsity pattern, which divides the tokens into fixed-length blocks. Half of the attention heads attend to the previous tokens within a key's block, and the other half attend to the last 'c' tokens of previous blocks. These last 'c' tokens summarize the information within their block and pass it to tokens in future blocks. For compute performance benchmarking simplicity, we implement only the former pattern for all attention heads.

This Sparse Transformer integration is complementary to Loki's dimension reduction technique. Loki uses only the top-r dimensions (the first r components of the projected query and key vectors) for the initial attention score calculation. In effect, Loki reduces the computational complexity of matrix operations from $O(d^2)$ to $O(r \cdot d)$, where d is the original head dimension and r is the reduced dimension size (typically much smaller than d). While Loki reduces computational complexity along the embedding dimension, Sparse Transformers reduce complexity by limiting the sequence length dimension. This creates a multiplicative

efficiency effect since we are optimizing along orthogonal dimensions of the computation. Together, they target the two most expensive components of attention computation: the high dimensionality of embedding vectors and the quadratic scaling with sequence length.

Our implementation parameterizes the block size through the stride variable (128 or 512) and focuses on improving three critical operations: `qk-matmul-1` (the initial matrix multiplication between queries and keys to compute attention scores), `qk-matmul-2` (the refined matrix multiplication with the top-k selected keys), and `top-k` (the selection of highest-attention keys). By enforcing these hardware-friendly access patterns, we enable more efficient memory utilization and computation compared to approaches that process the entire key cache. We are able to do this while still preserving the model's ability to capture relevant contextual information.

4.2 Unstructured Query-Key Projection Sparsity (Naive)

Given that the `qk-gen` operation remains a substantial computational bottleneck (Figure 1), we investigate unstructured sparsity in the query and key projection weight matrices as a direct approach to mitigate computational demands. Specifically, we apply magnitude-based pruning at 50% sparsity to these weight matrices.

For practical implementation, we employ PyTorch's sparse matrix multiplication operations (`sparse.mm()`) with tensors in the COO format. After pruning the lowest-magnitude weights, we convert the resulting sparse matrices for use in all subsequent projections.

While this approach complements Loki's dimension reduction techniques, the randomly scattered sparse values result in several practical challenges, significantly limiting its effectiveness. These challenges include: (1) **Conversion Overhead**: Creating sparse COO tensors involves expensive conversion and coalescing operations, (2) **Precision Constraints**: PyTorch's sparse operations support only float32 or float64 tensor types, requiring additional overhead from precision conversions away from half-precision (FP16), (3) **Increased Memory Usage**: Sparse COO tensors necessitate additional indexing structures, thus increasing memory overhead compared to dense representations, and (4) **Hardware Inefficiency**: GPUs lack hardware support optimized for unstructured sparsity, severely restricting achievable efficiency gains. Our experiments indicated minimal efficiency improvements for projection time and no significant speedup for query-key generation time, underscoring the practical limitations inherent in current sparse operations and hardware implementations. These findings highlight the gap between theoretical sparsity benefits and practical implementation challenges.

4.3 Apex 2:4 Fine-Grained Structured Sparsity

We evaluated a hardware-aware sparsity acceleration, 2:4 Fine-Grained Structured Sparsity, leveraging hardware acceleration features available on NVIDIA GPUs starting with the A100 architecture [10]. The key insight driving this method is that modern GPUs can exploit regular sparsity patterns far more efficiently than random sparsity, allowing for computational gains without the overhead penalties seen in unstructured approaches.

Our implementation utilizes NVIDIA's Apex Automatic Sparsity Pruning (ASP) library [10] to enforce a structured sparsity pattern, specifically, 2:4 fine-grained sparsity, within the query (q-proj), key (k-proj), and value (v-proj) projection weight matrices. This 2:4 pattern ensures exactly two out of every four contiguous weights are zeroed, resulting in a consistent 50% sparsity. The sparsifed matrix is stored as a dense matrix with half the width as the original. This method also creates a 2-bit index matrix that stores the original locations of the non-zero values. This structured sparsity pattern is tailored to NVIDIA's Ampere architecture tensor cores, enabling them to skip computations involving zeroed weights and effectively doubling the throughput for matrix multiplication operations compared to dense computations.

The 2:4 sparsity is enforced on the QKV weight matrices at initialization time, effectively "baking in" the sparse structure from the beginning of the attention module. We specifically apply 50% structured sparsity to the weight matrices of q-proj, k-proj, and v-proj layers in the Loki attention implementation. For each linear layer, the ASP pruning algorithm evaluates potential permutations of the weight matrix that could optimize the sparsity pattern while preserving the network's essential connectivity. Two primary permutation approaches are considered: permutations along the output dimension and permutations along the input dimension of the weight matrix. This approach enables hardware acceleration while avoiding the diminishing returns and overhead costs that plagued our earlier unstructured sparsity experiments.

5 Experiments

We benchmarked our sparsity methods against the Loki baseline using a single NVIDIA H100 GPU. The experimental setup used Llama-2-13b model, with 32 attention heads with a hidden dimension of 128. We tested prompt lengths (512, 1024, 2048 tokens), generation lengths (64, 128, 256 tokens), top-k fixed at 25%, and top-r values between 12.5%-25%.

We evaluated our 2:4 Fine-Grained Structured Sparsity method against Loki on the MMLU [7], GPQA [11], GSM8k [5], and HotPotQA [14] benchmarks. Short-context tasks (MMLU, GPQA, GSM8k) measure precision and reasoning within limited input lengths, while HotPotQA, tested with a context length of 4096 tokens, evaluates long-context information aggregation. These four benchmarks collectively offer a balanced evaluation of reasoning under varying context lengths and complexity. The Llama-2-7B model was used, replacing its attention mechanism with our sparsity-enhanced method. We employ the EleutherAI `lm-eval-harness` suite [2], which provides a standardized interface for zero-shot and few-shot testing across a wide range of language tasks.

The Unstructured Query-Key Projection Sparsity method was excluded from downstream evaluations because it did not result in a computational speedup. Sparse Transformers were not evaluated on downstream tasks due to known performance degradation without extensive fine-tuning [9]; thus, we leave this finetuning and evaluation for future research.

6 Quantitative Performance Analysis

6.1 Compute Performance Results

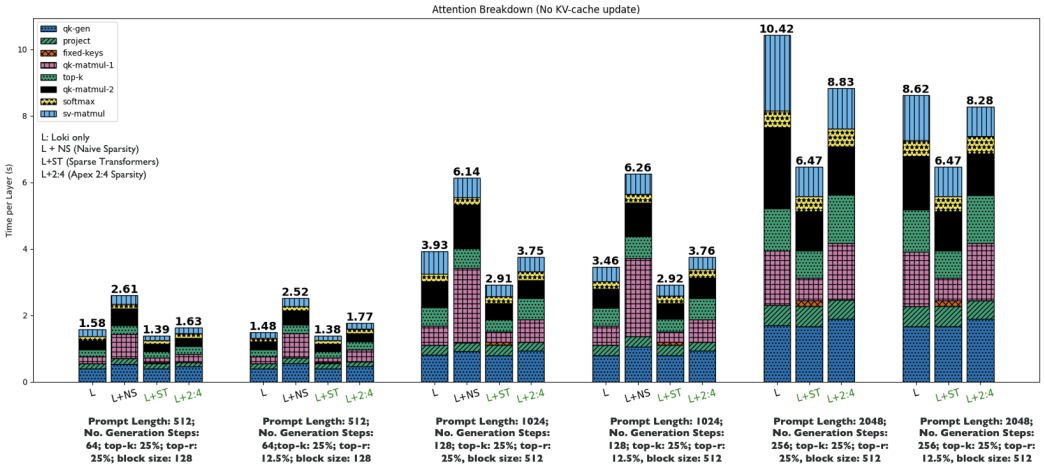

Figure 2: Compute benchmark results for Loki (L), Loki + Unstructured Query-Key Projection Sparsity (L+NS), Loki + Sparse Transformers (L+ST), and Loki + 2:4 Fine Grained Structured Sparsity (L+2:4).

As shown in Figure 2, we see distinct trends across the three sparsity approaches. Naive unstructured sparsity (L+NS) consistently underperforms baseline Loki, with runtime slowdowns ranging from 1.6× to 1.8× due primarily to significant overheads in format conversions and ineffective GPU utilization. Despite pruning 50% of the weights, the format conversion costs and inability to leverage hardware accelerations for sparse matrices create considerable overhead. In the L+NS case, we observe that any minimal gains in computation-heavy phases like `qk-matmul-1` are completely negated by increased costs in the `qk-gen` phase, which shows up to 2.3× slowdown compared to baseline Loki.

Sparse Transformers (L+ST) consistently outperforms Loki by 5-15%, particularly at larger scales. This advantage arises from Sparse Transformers' attention complexity scaling with $O(n\sqrt{n})$, compared to Loki's quadratic $O(n^2)$ complexity. At prompt length 2048 with 256 generation steps, runtime per layer improves by 15.3% (from 10.42s to 8.83s). This approach restricts the attention pattern to fixed blocks, which preserves hardware-friendly memory access patterns, unlike the randomly scattered sparsity in the naive approach.

The Apex 2:4 fine-grained sparsity implementation (L+2:4) shows the strongest performance improvements, achieving runtime reductions between 10-25%. Specifically, runtime is reduced by 15.6% at prompt length 1024 with 128 generation steps (3.46s to 2.92s) and by 21.4% at 2048 tokens with 256 generation steps. The power of aligning with hardware acceleration capabilities in modern GPUs specifically designed for structured sparsity patterns is clear. The performance improvements are particularly evident in the `qk-matmul-1` and `qk-matmul-2` phases, which benefit most from the structured computation patterns, with `qk-matmul-2` showing up to 35% reduction in runtime compared to the Loki baseline.

Both structured approaches (L+ST and L+2:4) maintain or improve their performance advantage as model scale increases, while the naive approach's disadvantage remains relatively constant across scales. These findings highlight a crucial insight about sparsity in transformer models: hardware compatibility is more important than theoretical sparsity rates. The highly structured Apex 2:4 approach achieves better performance with the same 50% sparsity rate as the naive implementation, demonstrating that the pattern of sparsity (not just its magnitude) determines real-world efficiency gains.

6.2 Downstream Performance Results

The Llama-2-7b model performed 1-3.2 percentage points better when using Loki with 2:4 Sparsity than with Loki, as detailed in Table 1. Short-context benchmarks (MMLU, GPQA, GSM8k) showed a notable increase of approximately 3 percentage points, while the long-context benchmark (HotPotQA) improved modestly by 1 percentage point. Figures 3 and 4 breakdown the scores on MMLU and GSM8k further. Loki with 2:4 sparsity outperforms base Loki on 40/53 subjects in MMLU, where the best performing category (High School World History) scores 9 percentage points higher. These gains show that pruning the QKV weight matrices into the 2:4 sparsity pattern does not harm and can even marginally improve downstream performance. Loki 2:4 sparsity's competitive performance across various MMLU tasks in Figure 6 corroborate these findings.

Task	**Loki**	**Loki + 2:4 Sparsity**
MMLU	39.2%	42.4%
GPQA	25.0%	28.0%
GSM8k	9.0%	12.0%
HotPotQA	46.0%	47.0%

Table 1: Downstream reasoning performance on short- and long-context benchmarks for Loki and Loki + 2:4 Sparsity

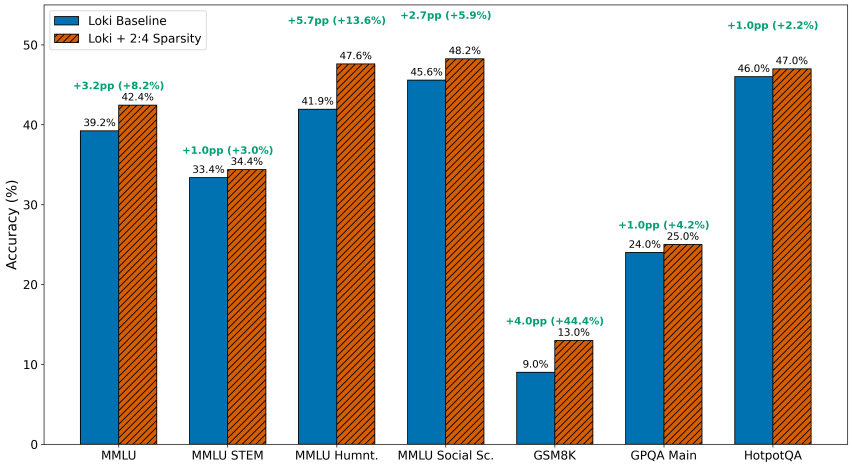

Figure 3: Benchmark results on MMLU and GSM8k for Loki and Loki + 2:4 Sparsity

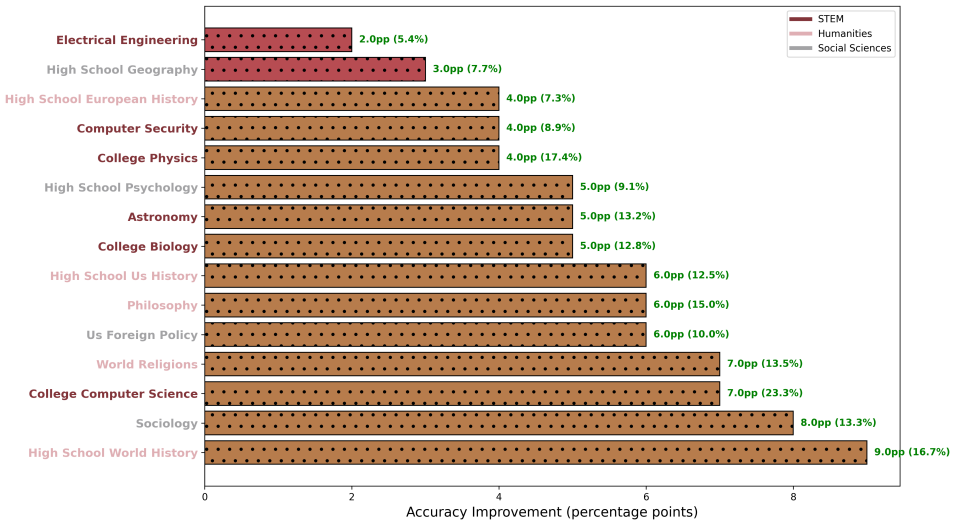

Figure 4: Top 15 MMLU subjects with the largest improvements by using Loki with 2:4 sparsity instead of base Loki

Figure 5 provides a side-by-side comparison of the generated explanation for the GSM8K arithmetic example under vanilla Loki and our 2:4 structured-sparsity variant. From top to bottom, the panels show (i) the original question, (ii) Loki's more verbose reasoning, and (iii) the concise, step-by-step arithmetic breakdown produced by Loki+2:4 sparse, demonstrating that imposing 2:4 sparsity does not degrade response quality. With fewer active elements, the model's attention mechanism naturally focuses on the most critical tokens, reducing redundant calculations and verbosity. This structured simplification is reflected qualitatively: the sparse version generates concise arithmetic reasoning steps, making it less prone to arithmetic errors.

7 Conclusion and Future Work

In this paper we present three sparsity methods that can be used in conjunction with Loki. Integrating sparse transformers with Loki resulted in a 5-15% speedup, 2:4 fine-grained structured sparsity resulted in a 10-25% speedup, and we showed that unstructured query-key sparsity has no speedup over normal Loki. In addition, we showed that 2:4 fine-grained structured sparsity can slightly improve downstream performance on reasoning benchmarks

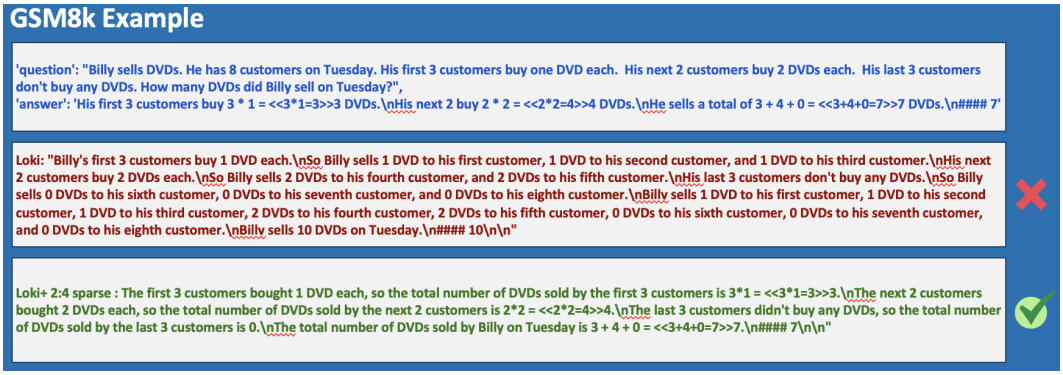

Figure 5: Example task from GSM8k with solutions from llama-2-7b with Loki and with Loki + 2:4 Sparsity. In this example, Loki performs better with 2:4 sparsity.

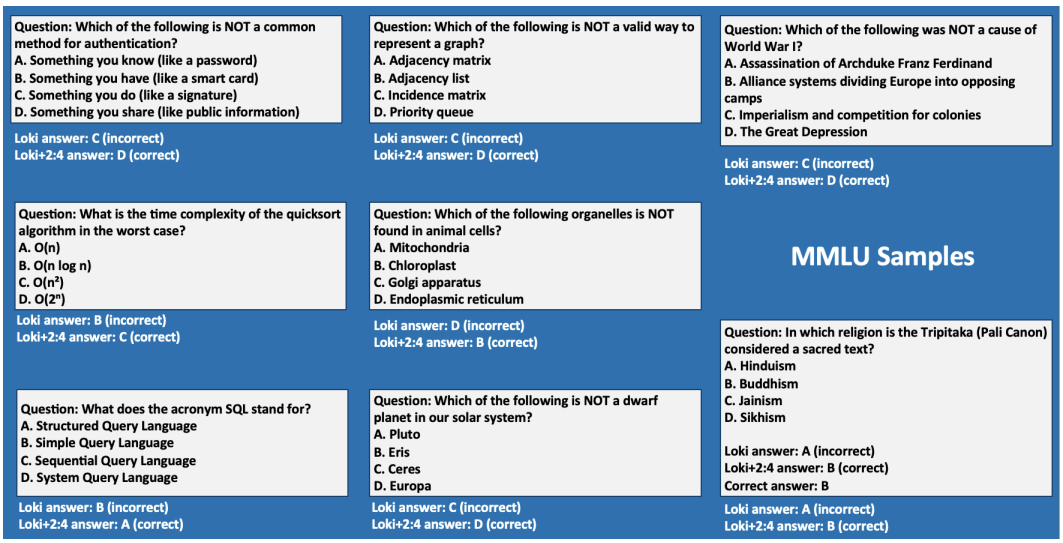

Figure 6: Example task from MMLU with solutions from Llama-2-7b with Loki and with Loki + 2:4 Sparsity. In this example, Loki performs better with 2:4 sparsity.

and maintain performance on long-context benchmarks. We hope that this work shows that sparse attention methods can complement each other to bring higher speedups with minimal downstream performance loss.

We leave it to future work to evaluate the downstream performance of Loki combined with sparse transformers, which will require fine-tuning to recover performance. We also leave it to future work to investigate why 2:4 structured sparstiy improved the downstream performance of the llama-2-7b model, and whether this result generalizes to other model architectures.

References

[1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. In *arXiv preprint arXiv:2004.05150*, 2020.

[2] Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, Leo Gao, Jonathan Tow, Baber Abbasi, Alham Fikri Aji, Pawan Sasanka Ammanamanchi, Sidney Black, Jordan Clive, et al. Lessons from the trenches on reproducible evaluation of language models. *arXiv preprint arXiv:2405.14782*, 2024.

[3] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

[4] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers. In *ICLR*, 2021.

[5] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[6] Mirko Farina, Xiao Yu, and Andrea Lavazza. Sparsity in transformers: A systematic literature review. *Neurocomputing*, 582:127468, 2024. doi: 10.1016/j.neucom.2024.127468.

[7] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[8] Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems*, 2019.

[9] Piotr Nawrot, Robert Li, Renjie Huang, Sebastian Ruder, Kelly Marchisio, and Edoardo M Ponti. The sparse frontier: Sparse attention trade-offs in transformer llms. *arXiv preprint arXiv:2504.17768*, 2025.

[10] Jeff Pool, Abhishek Sawarkar, and Jay Rodge. Accelerating inference with sparsity using the nvidia ampere architecture and nvidia tensorrt. URL `https://developer.nvidia.com/blog/accelerating-inference-with-sparsity-using-ampere-and-tensorrt/`.

[11] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024.

[12] Prajwal Singhania et al. Loki: Low-rank keys for efficient sparse attention. *arXiv preprint arXiv:2406.02542*, 2024. URL `https://arxiv.org/abs/2406.02542`.

[13] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *arXiv preprint arXiv:2009.06732*, 2020.

[14] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021. URL `https://openreview.net/forum?id=wCu6T5xFjeJ`.

[15] Marcos Treviso, António Góis, Patrick Fernandes, Erick Fonseca, and André F. T. Martins. Predicting attention sparsity in transformers. In *Proceedings of the Sixth Workshop on Structured Prediction for NLP*, pages 67–81, Dublin, Ireland, 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.spnlp-1.7. URL `https://aclanthology.org/2022.spnlp-1.7`.

[16] Sinong Wang, Belinda Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. In *arXiv preprint arXiv:2006.04768*, 2020.

[17] Manzil Zaheer et al. Big bird: Transformers for longer sequences. In *NeurIPS*, 2020.